

Universidad Mariano Gálvez de Guatemala

Ingeniería en Sistemas

Programación III



Esdras Estuardo Pérez Villatoro-94902312626

Wilder Alfonso Castillo Salguero- 9490213708

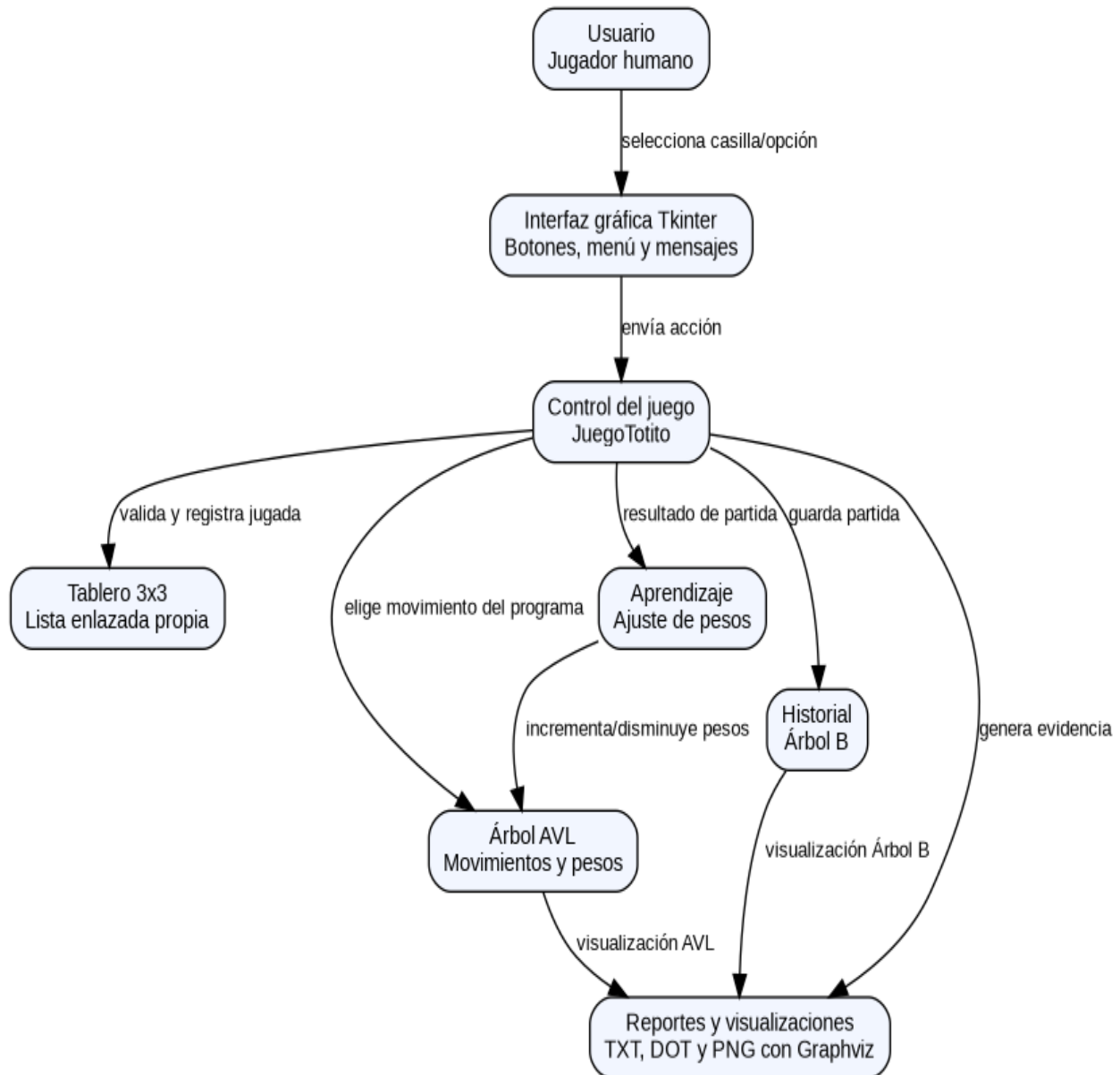
INDICE

1. Esquema conceptual del sistema	3
2. Arquitectura, componentes principales y flujo del sistema.....	4
3. Diagrama UML de clases.....	5
4. Descripción de clases	6
5. Descripción de las estructuras de datos implementadas	7
5.1 Lista enlazada simple	7
5.2 Árbol AVL.....	7
5.3 Árbol B	8
5.4 Observación técnica sobre restricciones	8
6. Lógica del aprendizaje implementado	8
7. Flujo técnico de una partida	9
8. Instalación técnica y ejecución	10
9. Observaciones y recomendaciones.....	10

1. Esquema conceptual del sistema

El sistema implementa un juego de Totito/Tic Tac Toe en el que un usuario humano juega contra un programa. El programa selecciona sus movimientos utilizando reglas estratégicas, exploración aleatoria y un árbol AVL donde cada posición del tablero posee un peso. Al terminar cada partida, los pesos se ajustan según el resultado obtenido.

Esquema conceptual del sistema



2. Arquitectura, componentes principales y flujo del sistema

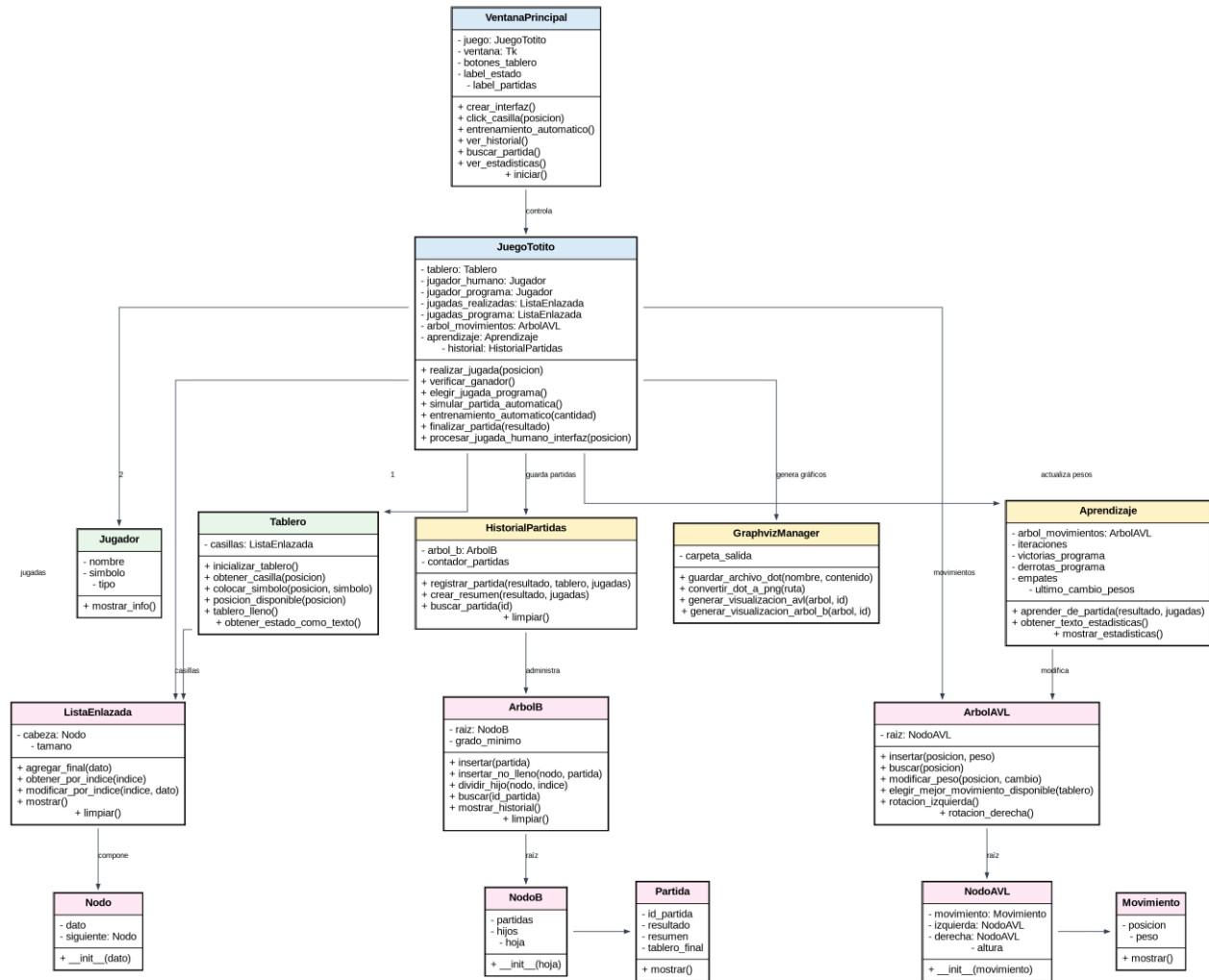
La arquitectura se encuentra organizada por paquetes dentro de la carpeta src. Cada paquete separa una responsabilidad del sistema: interfaz, juego, estructuras, aprendizaje, historial y utilidades.

Componente	Archivo/paquete	Responsabilidad
Punto de entrada	main.py	Crea la ventana principal e inicia el ciclo de ejecución de Tkinter.
Interfaz gráfica	src/interfaz/ventana_principal.py	Presenta el tablero, menú principal, ventanas de mensaje y acciones disponibles.
Control del juego	src/juego/juego_totito.py	Administra turnos, jugadas, ganador, entrenamiento, historial y aprendizaje.
Tablero y jugadores	src/juego/tablero.py, jugador.py	Representan las casillas del tablero y los datos de cada jugador.
Estructuras propias	src/estructuras/	Implementa lista enlazada, árbol AVL y árbol B.
Aprendizaje	src/aprendizaje/aprendizaje.py	Ajusta pesos de movimientos y registra estadísticas.
Historial	src/historial/historial_partidas.py	Registra partidas dentro de un árbol B.
Visualización	src/utilidades/graphviz_manager.py	Genera archivos DOT y PNG para representar el AVL y el árbol B.

Flujo resumido: el usuario interactúa con Tkinter, VentanaPrincipal envía la acción a JuegoTotito, el tablero valida la posición, el programa responde con una jugada calculada, se verifica si existe ganador o empate, y al finalizar se actualizan pesos, historial y reportes.

3. Diagrama UML de clases

Diagrama UML de clases - Sistema Totito con Aprendizaje



4. Descripción de clases

Clase	Atributos principales	Métodos principales	Responsabilidad
VentanaPrincipal	juego, ventana, botones_tablero, label_estado, label_partidas	crear_interfaz, click_casilla, entrenamiento_automatico, ver_historial, buscar_partida, ver_estadisticas, reiniciar_aprendizaje, iniciar	Construye y controla la interfaz gráfica del sistema. Conecta los botones de la pantalla con la lógica del juego.
JuegoTotito	tablero, jugador_humano, jugador_programa, turno_actual, jugadas_realizadas, jugadas_programa, arbol_movimientos, aprendizaje, historial, graphviz_manager	realizar_jugada, verificar_ganador, elegir_jugada_programa, simular_partida_automatica, entrenamiento_automatico, finalizar_partida	Clase central. Coordina tablero, jugadores, reglas, aprendizaje, historial y reportes.
Tablero	casillas	inicializar_tablero, obtener_casilla, colocar_simbolo, posicion_disponible, tablero_lleno, obtener_estado_como_texto	Representa el tablero de 9 posiciones mediante lista enlazada, evitando matriz 3x3.
Jugador	nombre, simbolo, tipo	mostrar_info	Modelo simple para identificar al jugador humano y al programa.
ListaEnlazada	cabeza, tamano	agregar_final, obtener_por_indice, modificar_por_indice, mostrar, limpiar	Estructura lineal propia para guardar casillas y jugadas.
Nodo	dato, siguiente	__init__	Nodo base de la lista enlazada simple.
ArbolAVL	raiz	insertar, buscar, modificar_peso, elegir_mejor_movimiento_disponible, cargar_movimientos_iniciales, reiniciar	Guarda movimientos posibles ordenados por posición y conserva pesos usados por el aprendizaje.
NodoAVL	movimiento, izquierda, derecha, altura	__init__	Nodo del AVL. Mantiene referencias izquierda/derecha y altura para balanceo.
Movimiento	posicion, peso	mostrar	Representa una casilla jugable y su peso estratégico actual.
Aprendizaje	arbol_movimientos, iteraciones, victorias_programa, derrotas_programa, empates, primera_victoria_en	aprender_de_partida, obtener_texto_estadisticas, mostrar_estadisticas	Actualiza pesos del AVL según resultado de la partida y genera estadísticas.
HistorialPartidas	arbol_b, contador_partidas	registrar_partida, crear_resumen, buscar_partida, limpiar	Crea objetos Partida y los almacena en el árbol B.
ArbolB	raiz, grado_minimo	insertar, insertar_no_lleno, dividir_hijo, buscar, mostrar_historial, limpiar	Estructura de historial ordenado por ID de partida.
NodoB	partidas, hijos, hoja	__init__	Nodo del árbol B. Guarda múltiples partidas y referencias a hijos.
Partida	id_partida, resultado, resumen, tablero_final	mostrar	Entidad de historial con resultado y estado final del tablero.
GraphvizManager	carpeta_salida	generar_visualizacion_avl, generar_visualizacion_arbol_b, guardar_archivo_dot, convertir_dot_a_png	Genera archivos visuales para evidenciar las estructuras utilizadas.

5. Descripción de las estructuras de datos implementadas

5.1 Lista enlazada simple

Propósito dentro del sistema: almacenar el tablero, las jugadas realizadas y las posiciones usadas por el programa sin utilizar matrices como estructura principal.

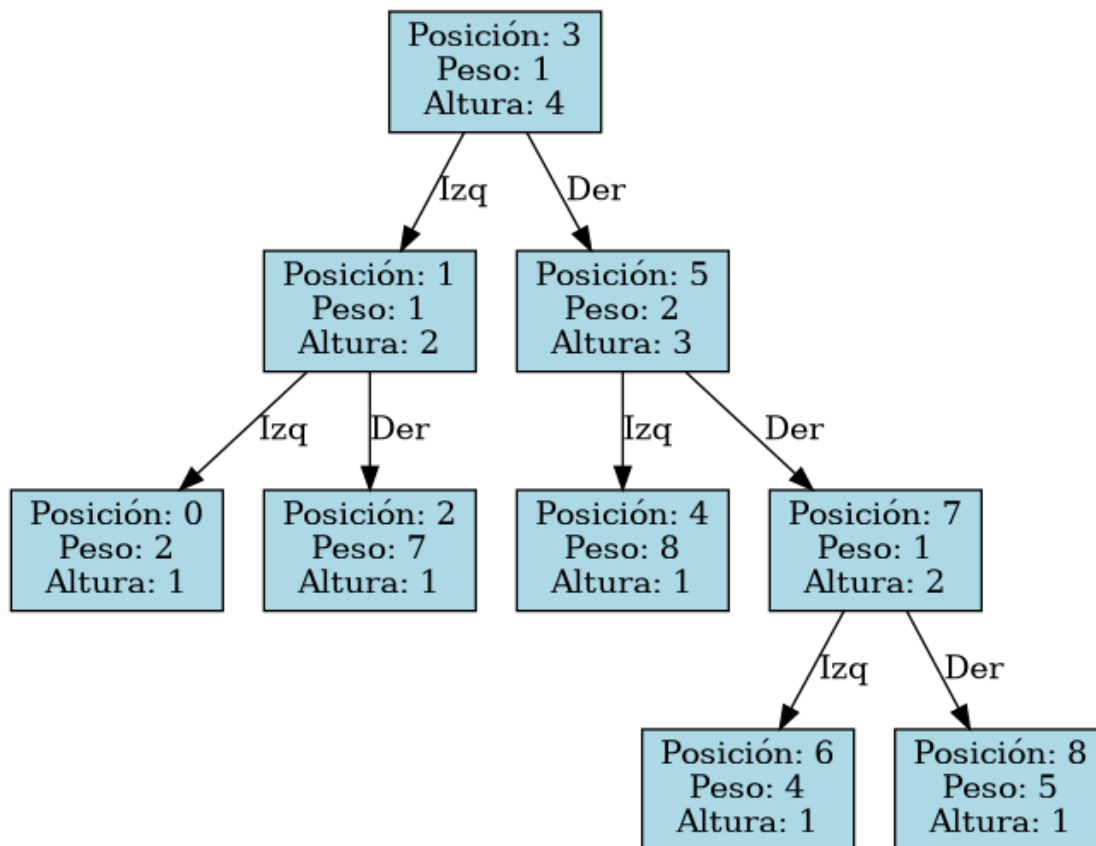
- Implementación: clase Nodo con dato y siguiente; clase ListaEnlazada con cabeza y tamaño.
- Operaciones: agregar al final, obtener por índice, modificar por índice, limpiar y mostrar.
- Justificación: permite guardar información secuencial y recorrerla nodo por nodo, cumpliendo la idea de estructuras propias.

5.2 Árbol AVL

Propósito dentro del sistema: almacenar los movimientos posibles del programa. Cada movimiento tiene una posición de tablero y un peso que representa su conveniencia.

- Implementación: Movimiento, NodoAVL y ArbolAVL. El árbol se ordena por posición y mantiene altura para balancearse.
- Operaciones: insertar, buscar, modificar peso, elegir mejor movimiento disponible, rotaciones izquierda/derecha y reinicio.
- Justificación: permite mantener movimientos ordenados y balanceados; además, el peso de cada posición sirve como memoria de aprendizaje.

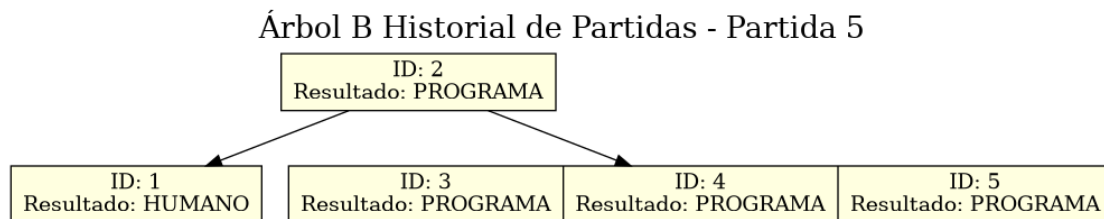
Árbol AVL de Movimientos - Partida 5



5.3 Árbol B

Propósito dentro del sistema: almacenar el historial de partidas por ID de forma ordenada y permitir búsqueda eficiente.

- Implementación: Partida, NodoB y ArbolB. El grado mínimo es configurable desde la interfaz.
- Operaciones: insertar partida, insertar en nodo no lleno, dividir hijo, buscar por ID, mostrar historial y limpiar.
- Justificación: el historial puede crecer con muchas partidas; el árbol B organiza registros por bloques y mantiene orden por ID.



5.4 Observación técnica sobre restricciones

El proyecto evita matrices para el tablero y utiliza estructuras propias para la lógica principal. Sin embargo, se observó que NodoB usa listas internas de Python para partidas e hijos, y VentanaPrincipal usa listas para organizar botones de la GUI. Si la restricción del curso prohíbe cualquier uso de listas, se recomienda reemplazar esas colecciones por una estructura propia especializada o justificar que el uso en la interfaz es auxiliar.

6. Lógica del aprendizaje implementado

El aprendizaje implementado es un aprendizaje por refuerzo simple basado en pesos. El programa conserva en el árbol AVL el peso de cada posición del tablero. Cuando termina una partida, la clase Aprendizaje recorre las jugadas realizadas por el programa y modifica los pesos según el resultado.

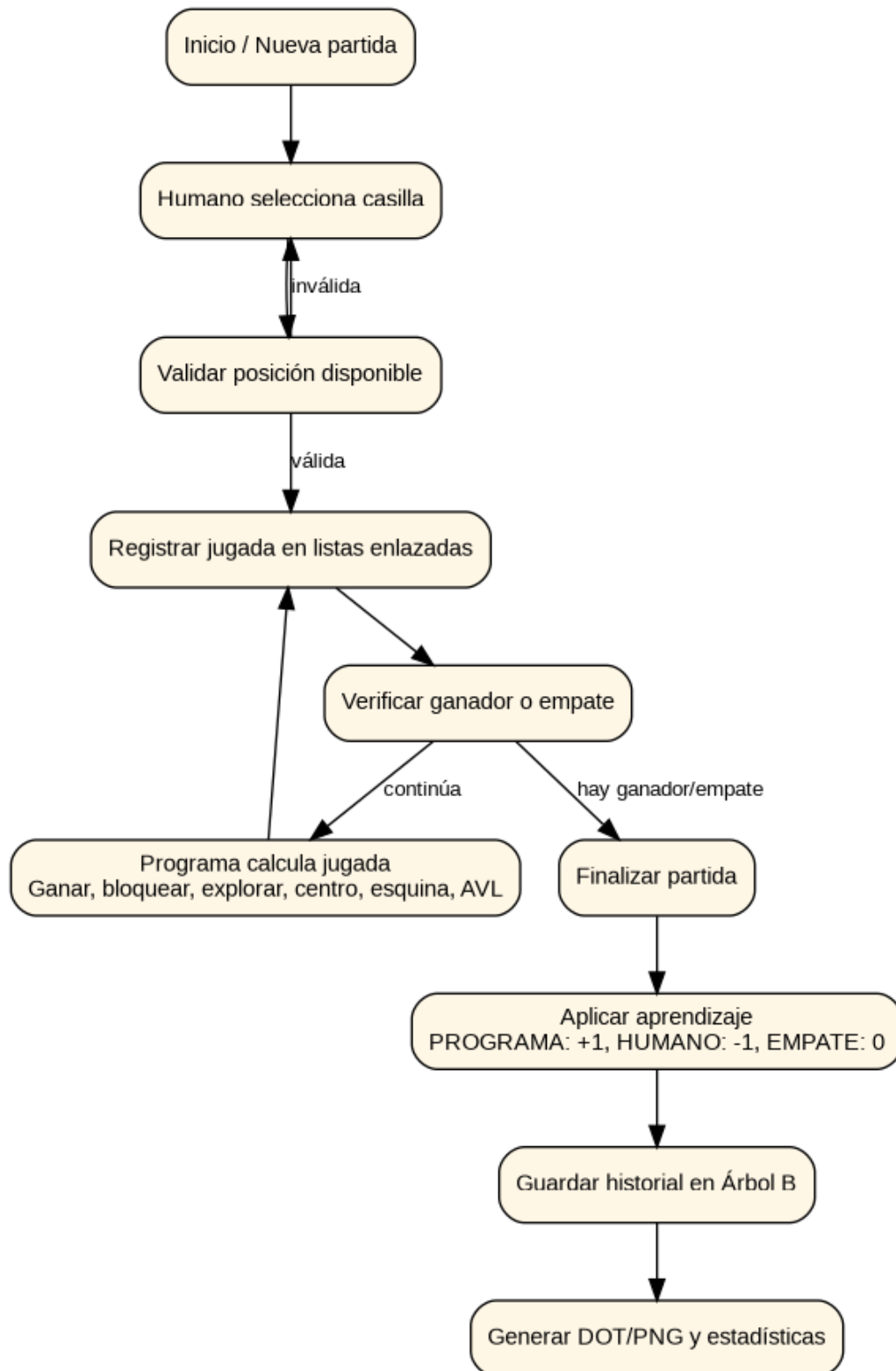
Resultado	Cambio aplicado	Efecto
Gana el programa	+1	Refuerza las posiciones usadas por el programa.
Gana el Usuario	-1	Penaliza las posiciones que llevaron a una derrota. El peso mínimo queda en 0.
Empate	0	No altera los pesos, pero registra el empate en estadísticas.

Además del AVL, la selección de jugada evoluciona por etapas. Al inicio el programa explora más y bloquea menos; conforme aumentan las iteraciones, suben las probabilidades de ganar, bloquear, tomar el centro y tomar esquinas. La exploración disminuye gradualmente.

Iteraciones	Comportamiento aproximado
0 a 9	Mayor exploración y estrategia básica.
10 a 39	Mejora bloqueo, centro y esquinas.
40 a 99	Mayor prioridad a jugadas ganadoras y bloqueo.
100 o más	Menor exploración y decisión más estratégica.

7. Flujo técnico de una partida

Flujo general de una partida



1. El usuario inicia una nueva partida desde la interfaz.
2. VentanaPrincipal llama a JuegoTotito para procesar la jugada del humano.
3. Tablero valida si la casilla está libre y registra el símbolo X.
4. JuegoTotito verifica ganador o empate.
5. Si la partida continúa, el programa calcula una jugada con prioridad de ganar, bloquear, explorar, centro, esquina y finalmente AVL por peso.
6. La jugada del programa se registra en la lista enlazada de jugadas y en la lista de jugadas del programa.
7. Al finalizar, Aprendizaje ajusta pesos en el AVL según el resultado.
8. HistorialPartidas guarda la partida en el árbol B.
9. GraphvizManager genera archivos DOT y PNG del AVL y del árbol B.

8. Instalación técnica y ejecución

Requisitos sugeridos para desarrollo: Python 3.10 o superior, Tkinter disponible en la instalación de Python y Graphviz instalado para generar imágenes PNG de las estructuras.

Paso	Comando/acción
1	Descomprimir el archivo Proyecto-Final-main.zip.
2	Abrir una terminal en la carpeta Proyecto-Final-main.
3	Ejecutar: <code>python main.py</code>
4	Opcional para Graphviz: instalar Graphviz y verificar que el comando dot esté disponible.
5	Los reportes se generan en la carpeta reportes y las visualizaciones en <code>imagenes_graphviz</code> .

El archivo requerimientos.txt se encuentra vacío porque el proyecto usa librerías estándar de Python.

Graphviz no es una librería Python obligatoria, pero sí una herramienta externa necesaria para convertir DOT a PNG.

9. Observaciones y recomendaciones

- Completar README.md con integrantes, carné y porcentaje de participación.
- Agregar al colaborador ingVillatoroUMG en el repositorio, según los lineamientos del proyecto.
- Eliminar el método duplicado `buscar_esquina_disponible` en `juego_totito.py` para mantener el código más limpio.
- Revisar el uso de listas Python en NodoB y botones de interfaz si el evaluador aplica la restricción de manera estricta.
- Documentar en el repositorio cómo ejecutar el sistema y cómo probar entrenamiento, historial y Graphviz.